Chapter 01

Host Preparation & Hypervisor Setup

Static IPs. Consistent hostnames. A planned topology decided before a single server boots — the blueprint phase that saves ten times the time later.

12	10.0.2.0/24	UTC	8+
VMS CLONED	LAB NETWORK	TIME STANDARD	HARDENING STEPS

1.1 WHY THIS STEP MATTERS

Plan Before You Provision

Most people skip straight to installing tools — then spend days troubleshooting IPs that keep changing.

❌ The Common Mistake

- ✗ Create VMs with default settings
- ✗ Use whatever network the hypervisor assigns
- ✗ Discover two services can't talk — IPs keep changing
- ✗ Troubleshoot for days instead of planning for hours

✔️ The Professional Approach

- ✓ Decide addressing, naming & topology first
- ✓ Plan before a single server boots
- ✓ Static IPs and consistent hostnames from day one
- ✓ Infrastructure planning happens before provisioning

🏗️ **Analogy — The Architectural Drawing Phase:** You don't pour concrete before you know where the walls go. A well-planned network is a well-drawn blueprint — it saves ten times the time later when integrating a dozen services.

💡 **Key Takeaway:** *Static IPs, consistent hostnames, and a planned network topology are not optional niceties — they are prerequisites for a reliable lab.*

Choosing the Right Hypervisor

Hypervisor	Type	Cost	Best For
KVM/QEMU + libvirt	Type-1 (Linux)	Free	*Production-like, best performance*
VirtualBox	Type-2	Free	*Beginners, quick setup*
VMware Workstation/Fusion	Type-2	Paid	*Advanced networking, snapshots*
Proxmox VE	Type-1 (bare metal)	Free	*Dedicated lab server, web UI*
UTM (Apple Silicon)	Type-2 (QEMU)	Free	*M1/M2/M3 Mac users*

This lab uses KVM/QEMU on Linux — lowest overhead, scriptable virsh/virt-install (matters for Packer/OpenTofu), and the professional DevOps standard.

HARDWARE REQUIREMENTS



CPU

8 cores min · 16+ recommended



RAM

32 GB min · 64 GB recommended



Disk

500 GB SSD min · 1 TB NVMe rec.



Network

1 Gbps, wired preferred

Check virtualization support:

```
grep -E '(vmx|svm)' \
  /proc/cpuinfo | head -3
```

Step 1–2: Update the Host, Install KVM

Step 1 — Update the host system

```
sudo apt update && sudo apt upgrade -y
```

🔐 *An unpatched host is not a security lab — it's a liability.*

Step 2 — Install KVM and management tools

```
sudo apt install -y \
    qemu-kvm \
    libvirt-daemon-system \
    libvirt-clients \
    bridge-utils \
    virt-manager \
    virtinst \
    cpu-checker \
    guestfs-tools
```

WHAT EACH PACKAGE DOES

qemu-kvm	The hypervisor & hardware emulation engine
libvirt-daemon-system	Background daemon managing VMs via API
libvirt-clients	virsh — CLI to manage VMs
bridge-utils	Create & manage network bridges
virt-manager	Graphical VM manager
virtinst	virt-install — create VMs from CLI
cpu-checker	kvm-ok — verifies CPU supports KVM
guestfs-tools	Inspect/modify disk images without booting

Step 3–5: Verify, Authorize, Enable

3 Verify KVM is working

```
$ kvm-ok
→ INFO: /dev/kvm exists
→ KVM acceleration can be used
```

4 Add your user to libvirt group

```
$ sudo usermod -aG libvirt,kvm $USER
$ newgrp libvirt
```

5 Enable & verify libvirtd

```
$ sudo systemctl enable --now
libvirtd
$ virsh list --all
→  Id   Name   State
→ ----------------------
```

Why this matters: Managing VMs as your normal user (not root) follows least privilege — a compromised session doesn't mean instant root on the host.

Choosing the Right Network Mode

Mode	→ Host	→ Other VMs	→ Internet	Use For
NAT (default)	Yes	Yes	Yes (via NAT)	*Simple testing*
Bridged	Yes	Yes	Yes (direct)	*Production-like*
Host-only	Yes	Yes	No	*Isolated lab* ✅

We use a combination

Host-only / isolated network (10.0.2.0/24): Primary lab network. All VMs communicate here.

NAT bridge: Optional, temporary — for package downloads during setup only.

🏗️ Analogy — The Office Building

The host machine is the building and the router at the entrance. Each VM is a room inside. The network we design here is the internal wiring — who can reach whom, and through what doors.

Creating the Lab Network

```
# Define the network in libvirt
$ virsh net-define /tmp/lab-network.xml
→ Network devsecops-lab defined

# Start it, then set it to auto-start on boot
$ virsh net-start devsecops-lab
$ virsh net-autostart devsecops-lab

# Verify it's active
$ virsh net-list --all
→  Name           State    Autostart
→  devsecops-lab   active   yes

# Verify the bridge on the host
$ ip addr show virbr-lab
→ inet 10.0.2.1/24 brd 10.0.2.255 ...
```



Key Takeaway

`10.0.2.1` is now your host machine's IP on the lab network.

Every VM on devsecops-lab can reach the host. The host can reach every VM.

VMs cannot reach the internet by default — exactly what a security lab wants.

One ISO, Downloaded and Verified Once

Every VM in this lab runs Ubuntu 24.04 LTS Server — a single verified source of truth.

1. Download

Ubuntu 24.04 Server ISO — no GUI, smaller attack surface

2. GPG Verify

Confirm Ubuntu signed the checksum file — not an attacker

3. Checksum Verify

sha256sum confirms the ISO matches, byte for byte

4. Trusted ISO

Single source of truth for every VM in the lab

```
$ wget https://releases.ubuntu.com/.../ubuntu-24.04.4-live-server-amd64.iso
$ gpg --verify SHA256SUMS.gpg SHA256SUMS
→ gpg: Good signature from "Ubuntu CD Image Automatic Signing Key"
$ sha256sum -c SHA256SUMS 2>/dev/null | grep ubuntu-24.04
→ ubuntu-24.04.4-live-server-amd64.iso: OK
```

🔐 Security Note — Chain of Trust

Verify Ubuntu signed the checksum, then verify the ISO matches it. A tampered ISO fails the checksum step. Skipping this is how supply-chain attacks succeed.

virt-install: One Command, Every VM Descends From It

```
$ virt-install \
  --name "ubuntu-24.04-base" \
  --ram 2048 \
  --vcpus 2 \
  --disk path=.../ubuntu-24.04-base.qcow2,\
     format=qcow2,size=20 \
  --os-variant ubuntu22.04 \
  --network network=devsecops-lab,\
     model=virtio \
  --graphics none \
  --console pty,target_type=serial \
  --cdrom .../ubuntu-24.04.4-live-server-amd64.iso
```

Why qcow2? Thin-provisioned — a 20 GB disk may only use 3 GB initially — and it supports snapshots, essential for rolling back to a clean lab state.

FLAG BREAKDOWN

`--name`	Template identifier
`--ram / --vcpus`	2 GB / 2 vCPU — enough for install
`--disk`	QCOW2, 20 GB, thin-provisioned
`--os-variant`	Optimal QEMU settings for this OS
`--network`	Our lab network; virtio = faster
`--graphics none`	No GUI — serial console only, more secure
`--console pty`	Attach terminal to the VM's serial port

The Common Toolset Every VM Needs

Networking & Diagnostics

`curl · wget · net-tools · dnsutils · tcpdump · nmap`

Security & Hardening

`ufw · fail2ban · auditd · gnupg · ca-certificates`

System & Scripting

`git · vim · htop · rsyslog · chrony · jq · python3`

Why chrony? Time sync is critical for TLS certificates and SIEM log correlation. **Why auditd?** System call auditing — required for compliance frameworks.

🔐 **Set the timezone — UTC, every VM, no exceptions**

```
timedatectl set-timezone UTC
```

If VMs use different timezones, log correlation in your SIEM becomes nearly impossible.

SSH Hardening, Firewall & Fail2ban

sshd_config Setting	Why
`PermitRootLogin no`	Always use a named account, never root
`PermitEmptyPasswords no`	No blank-password logins
`MaxAuthTries 3`	Limit brute-force login attempts
`AllowUsers devops`	Only the devops account may SSH in
`X11Forwarding no`	No GUI needed — reduces attack surface
`ClientAliveInterval 300`	Disconnect idle sessions after 10 min

UFW — Default Deny

```
ufw default deny incoming
ufw default allow outgoing
ufw allow 22/tcp
```

Every clone starts with SSH allowed, everything else blocked — defense-in-depth from the infrastructure up.

fail2ban: bans an IP for 1 hour after 5 failed SSH attempts in 10 minutes — automatic protection against brute-force attacks, active on every VM from first boot.

Strip Identity, Then Clone

Before shutting down the template

```
truncate -s 0 /etc/machine-id
rm -f /var/lib/dbus/machine-id
ln -s /etc/machine-id /var/lib/dbus/machine-id
rm -f /etc/ssh/ssh_host_*
history -c && history -w
shutdown -h now
```

Why remove machine-id? Every clone sharing one machine ID causes systemd/DHCP conflicts. An empty file makes each VM generate its own on first boot.



create-vm.sh — the clone script

```
./create-vm.sh <hostname> <ip> \
               <ram_mb> <vcpus> <disk_gb>
```

Given a base image, the script:

1. Clones the base qcow2 (copy-on-write, independent)
2. Defines the VM in libvirt with virt-install --import
3. Starts the VM automatically
4. Prints next steps: set hostname, static IP, DNS

One script, twelve invocations — the moment the lab comes alive.

Twelve VMs, One Command Each

Hostname	IP	RAM	vCPU	Disk
sec-dns-01	10.0.2.10	2048	2	20
sec-idm-01	10.0.2.11	6144	4	40
sec-edge-01	10.0.2.12	4096	4	30
sec-vault-01	10.0.2.13	4096	4	40
sec-vcs-01	10.0.2.14	8192	8	80
sec-appsec-01	10.0.2.15	8192	4	60

Hostname	IP	RAM	vCPU	Disk
sec-registry-01	10.0.2.16	8192	4	100
k8s-master-01	10.0.2.20	6144	4	60
k8s-worker-01	10.0.2.21	8192	4	80
k8s-worker-02	10.0.2.22	8192	4	80
sec-obs-01	10.0.2.30	12288	6	120
sec-siem-01	10.0.2.31	8192	4	80

```
$ ./create-vm.sh sec-dns-01 10.0.2.10 2048 2 20
```

Same pattern, 12 times — clone, define, start. Each VM boots on devsecops-lab, ready for hostname and static IP configuration.

Netplan: One Pattern, Every VM

```
# /etc/netplan/00-lab-network.yaml
network:
  version: 2
  renderer: networkd
  ethernets:
    enp1s0:
      dhcp4: false
      addresses:
        - 10.0.2.10/24
      routes:
        - to: default
          via: 10.0.2.1
      nameservers:
        addresses:
          - 10.0.2.10
          - 8.8.8.8
        search:
          - devsecops.com
```

Apply with: netplan apply → verify with: ip addr show enp1s0

STATIC IP REFERENCE

sec-dns-01	10.0.2.10	sec-registry-01	10.0.2.16
sec-idm-01	10.0.2.11	k8s-master-01	10.0.2.20
sec-edge-01	10.0.2.12	k8s-worker-01	10.0.2.21
sec-vault-01	10.0.2.13	k8s-worker-02	10.0.2.22
sec-vcs-01	10.0.2.14	sec-obs-01	10.0.2.30
sec-appsec-01	10.0.2.15	sec-siem-01	10.0.2.31

Every VM uses 10.0.2.10 as primary DNS (once sec-dns-01 is running) with 8.8.8.8 as fallback.

One Key Pair, Twelve VMs

```
$ ssh-keygen -t ed25519 \
    -C "devsecops-lab-$(date +%Y)" -f ~/.ssh/devsecops-lab
```

Why Ed25519? Elliptic-curve crypto — smaller, faster to verify, and currently the most secure widely-supported SSH key type.

```
# ~/.ssh/config — connect by simple hostname
Host sec-dns-01
  HostName 10.0.2.10
  User devops
  IdentityFile ~/.ssh/devsecops-lab
```

```
# Distribute the key once per VM
for VM_IP in 10.0.2.10 10.0.2.11 ...; do
  ssh-copy-id -i ~/.ssh/devsecops-lab.pub devops@${VM_IP}
done
```



Test key-based auth

```
$ ssh sec-dns-01 'hostname && \
    ip addr show | grep "10.0.2"'
→ sec-dns-01
→     inet 10.0.2.10/24 ...
```

🔐 Security Note

The private key (~/.ssh/devsecops-lab) never leaves the host. Only the public key (.pub) is copied to VMs. Password authentication stays enabled temporarily until every VM has the key — then it gets disabled in Chapter 03.

Prove It Works Before Moving On

validate-chapter-01.sh — checks

- **Hypervisor**

  KVM module loaded · libvirtd active · lab network active

- **Network Bridge**

  virbr-lab exists with 10.0.2.1 · IP forwarding enabled

- **VM Status**

  All 12 VMs exist and appear in virsh list --all

- **SSH Connectivity**

  Key-based SSH succeeds to sec-dns-01, idm-01, edge-01

🎉 *All checks passed! Ready for Chapter 02.*

Common Problems

kvm-ok fails

Hardware virtualization disabled in BIOS — enable VT-x/AMD-V, reboot

net-start: "already exists"

virsh net-destroy then net-undefine, then redefine and start

VM has no network after cloning

Interface name differs — check ip link show, fix netplan

ssh-copy-id: connection refused

VM not fully booted yet — check virsh list --all, wait, retry

cloud-init errors on boot

Disable it: systemctl disable cloud-init; not used in this lab

1.12 CHAPTER SUMMARY

What We Built — And Why It Matters

- **KVM/QEMU + libvirt**
  Production-grade, scriptable hypervisor
- **Base Ubuntu VM with hardening**
  Clone once, configure per role
- **Static IPs matching architecture**
  Services find each other reliably
- **Isolated lab network (10.0.2.0/24)**
  Controlled, predictable networking
- **SSH hardening on every VM**
  Defense-in-depth from the start
- **SSH key authentication**
  Passwords aren't scalable; keys are auditable

💡 The Meta-Lesson — Remove Ambiguity

Static IPs remove IP ambiguity. SSH keys remove authentication ambiguity. UTC removes time ambiguity. A consistent base image removes configuration ambiguity. Professional infrastructure is boring, predictable, and correct — exciting infrastructure is usually broken infrastructure.

Next: Chapter 02 *— VM Provisioning Strategy: turning this manual process into a repeatable, documented workflow.*